

Campanha de Simulação de Phishing

Documentação Técnica do Projeto — Página de Conscientização + Rastreo

Empresa: Diretriz / Dediccar • Uso interno e autorizado

Atualizado em: 10/07/2026

1. O que é este projeto (visão geral)

Este é um projeto de **conscientização em segurança da informação**. A empresa autorizou uma campanha interna de *phishing simulado*: um e-mail parecido com um golpe real é enviado aos colaboradores. Quem **clica no link** não cai em nenhuma armadilha de verdade — é levado a uma página que explica, de forma amigável, que aquilo era um teste e ensina como reconhecer golpes reais.

O projeto tem **duas partes** que trabalham juntas:

Parte	O que faz	Onde vive
1. Página estática (front-end)	A tela que o colaborador vê quando clica no link. Mostra o aviso "você participou de uma simulação" e as dicas de segurança.	<code>index.html</code> , <code>css/</code> , <code>js/</code>
2. Rastreo (back-end)	Conta, de forma anônima, quantas pessoas clicaram e quantas leram as orientações. Guarda os números num banco.	<code>api/</code> , <code>scripts/</code>

Analogia: pense na página como um *cartaz educativo* colado atrás de uma porta. O rastreo é apenas um *contador de pessoas* que passaram pela porta — ele conta quantas passaram, mas o cartaz em si não pede nome, senha nem dado nenhum de quem está lendo.

2. Tecnologias usadas (e por quê)

Tecnologia	Para que serve	Por que foi escolhida
HTML5	Estrutura e conteúdo da página (textos, títulos, botão).	É a base de qualquer site. Para uma página simples, não precisamos de mais nada.
CSS3	Aparência: cores, espaçamento, layout, modo escuro.	Separa o "visual" do "conteúdo" — boa prática que deixa o código organizado.
JavaScript (vanilla)	Interatividade: o botão "Li e entendi" e o envio do rastreo.	"Vanilla" = JS puro, sem framework. Para tão pouca interação, React/Vue seriam exagero.

Vercel	Hospeda o site na internet e roda o back-end.	Deploy grátis, automático a cada <code>git push</code> , e suporta as "funções" do back-end.
Redis (via <code>ioredis</code>)	Banco de dados que guarda os números de cliques/leituras.	É rápido e simples para contadores. A biblioteca <code>ioredis</code> conecta o back-end ao banco.
Git + GitHub	Controle de versão e onde o código fica guardado.	Guarda o histórico de tudo e a Vercel puxa o código direto do GitHub.

3. Estrutura de pastas

```

Pishing/
├── index.html      → a página que o colaborador vê
├── css/
│   └── style.css   → todo o visual da página
├── js/
│   └── script.js   → o botão e o envio do rastreo (roda no navegador)
├── api/            → o back-end (roda nos servidores da Vercel)
│   ├── track.js    → RECEBE e grava os cliques/leituras
│   └── stats.js     → MOSTRA os números (protegido por senha)
├── scripts/        → ferramentas que você roda no seu PC
│   ├── relatorio-cliques.mjs → cruza "quem clicou" com os e-mails
│   └── limpar-dados.mjs    → zera o placar antes de uma nova campanha
├── package.json    → lista as bibliotecas que o projeto usa
├── .gitignore       → arquivos que NÃO vão para o GitHub (ex.: senhas)
├── README.md        → instruções rápidas do projeto
└── CLAUDE.md        → instruções para o assistente de IA

```

Por que separar em pastas? Cada tipo de coisa fica no seu lugar: HTML (conteúdo), CSS (visual) e JS (comportamento). Isso se chama *separação de responsabilidades* — facilita achar e consertar as coisas depois. É como uma cozinha organizada: temperos num pote, talheres numa gaveta.

4. A página (front-end)

4.1. index.html — o esqueleto

O HTML é dividido em três blocos visuais:

- `<header class="alerta">` — a faixa de aviso no topo, com o ícone  e o título.
- `<main class="conteudo">` — o "cartão" central com a explicação e a lista de dicas.
- `<footer class="rodape">` — os contatos da equipe de Segurança e LGPD.

Dois detalhes importantes no `<head>`:

- `<meta name="robots" content="noindex, nofollow">` — pede ao Google para **não** indexar a página. Faz sentido: é uma página interna, não queremos que apareça em buscas.

- `<script src="js/script.js" defer>` — o `defer` faz o JavaScript carregar só **depois** que a página terminar de montar. Isso evita o erro clássico de o JS tentar mexer num botão que ainda não existe na tela.

4.2. css/style.css — o visual

Alguns pontos que valem entender:

- **Variáveis CSS** (o bloco `:root { --color-bg: ... }`): guardam as cores num único lugar. Se quiser mudar o azul do site, muda em *um* lugar só, e não em 20.
- **Modo escuro** (`@media (prefers-color-scheme: dark)`): detecta se o computador do colaborador está no tema escuro e troca as cores automaticamente.
- **clamp()** (ex.: `font-size: clamp(1.4rem, 4vw, 2rem)`): faz o texto crescer/encolher conforme o tamanho da tela, sem ficar nem minúsculo no celular nem gigante no monitor.

4.3. js/script.js — a interatividade e o rastreo

Este arquivo faz três coisas, na ordem:

1. **Descobre quem é a pessoa:** lê o `?rid=...` da URL. O GoPhish (ferramenta que dispara os e-mails) coloca nesse `rid` um código que representa cada destinatário. Se não vier nada, usa `"sem-id"`.
2. **Registra o clique:** assim que a página abre, chama a função `rastrear('clique')`, que envia esse evento ao back-end.
3. **Registra a leitura:** quando a pessoa clica no botão "Li e entendi", mostra a mensagem de agradecimento e envia `rastrear('leu')`.

```
function rastrear(evento) {
  fetch('/api/track', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ evento, id }),
    keepalive: true,
  }).catch(() => {});
}
```

Por que o `.catch(() => {})` vazio? O rastreo é "best effort" (na melhor das tentativas). Se o envio falhar (internet ruim, por exemplo), **não queremos travar a página** nem mostrar erro para o colaborador. O objetivo principal é educar; o contador é secundário. O `keepalive: true` ajuda o envio a terminar mesmo se a pessoa fechar a aba logo em seguida.

5. O rastreo (back-end)

As pastas `api/` na Vercel viram automaticamente **URLs**. O arquivo `api/track.js` fica acessível em `/api/track`. Isso se chama *Serverless Function* (função sem servidor): você não gerencia servidor nenhum — a

Vercel liga a função quando alguém chama e desliga sozinha depois.

5.1. api/track.js — grava os eventos

Quando recebe um evento, ele guarda no Redis três informações:

Comando Redis	O que guarda	Para quê
<code>incr total:clique</code>	Um contador que só aumenta.	Total de cliques (conta repetições também).
<code>sadd pessoas:clique</code>	Um conjunto de <code>rid</code> s (sem repetir).	Quantas pessoas únicas clicaram.
<code>lpush log</code>	Uma lista dos últimos eventos com data.	Histórico recente (limitado a 1000 com <code>ltrim</code>).

Diferença esperta: o `incr` conta *cliques* (se a pessoa abrir 3 vezes, conta 3). O `sadd` conta *pessoas* (a mesma pessoa abrindo 3 vezes conta 1, porque um "conjunto" não aceita repetidos). Ter os dois números dá uma leitura mais honesta da campanha.

5.2. api/stats.js — mostra os números (com senha)

Este endpoint devolve o placar da campanha em formato JSON. A **primeira linha da lógica é a segurança**:

```
if (!process.env.STATS_SENHA || req.query.senha !== process.env.STATS_SENHA) {  
  return res.status(401).json({ erro: 'nao autorizado' });  
}
```

Ou seja: só devolve os dados se a URL trouxer `?senha=` com o valor certo, guardado numa *variável de ambiente* chamada `STATS_SENHA`. Sem isso, responde `401 (não autorizado)`. Assim, só quem sabe a senha vê o placar.

Variável de ambiente: é um "segredo" (senha, endereço do banco) que fica configurado *fora* do código, no painel da Vercel. **Nunca** escrevemos senhas direto no código — por isso o `.env.local` está no `.gitignore` e não vai para o GitHub.

5.3. scripts/ — ferramentas do seu PC

- `relatorio-cliques.mjs`: cruza os `rid`s que clicaram (vindos do Redis) com os e-mails/nomes (vindos do GoPhish, que roda no seu computador). Assim você descobre *quem* clicou. Importante: esse cruzamento acontece **só na sua máquina** — nenhum e-mail é enviado a serviço externo.
- `limpar-dados.mjs`: apaga o placar (cliques, leituras e log) para começar uma nova campanha do zero.
Atenção: é irreversível.

Ambos rodam com o comando (na raiz do projeto):

```
node --env-file=.env.local scripts/relatorio-cliques.mjs
```

6. Como o fluxo funciona (do clique ao relatório)

1. O **GoPhish** dispara o e-mail de teste, cada um com um link contendo um `rid` único.
2. O colaborador **clica** → o navegador abre `index.html` na Vercel.
3. O `script.js` chama `/api/track` e registra o **clique** no Redis.
4. A pessoa lê e clica em "**Li e entendi**" → registra o evento **leu**.
5. Você abre `/api/stats?senha=...` para ver o placar, ou roda o script de relatório no seu PC.

7. Como rodar e publicar

7.1. Testar localmente

1. Abra a pasta no VS Code.
2. Instale a extensão **Live Server**.
3. Clique com o botão direito em `index.html` → "**Open with Live Server**".

7.2. Publicar na Vercel (deploy)

1. Suba as mudanças para o GitHub: `git add .` → `git commit -m "mensagem"` → `git push`.
2. A Vercel percebe o `push` e **publica automaticamente** a nova versão.
3. No painel da Vercel, configure as variáveis de ambiente `REDIS_URL` e `STATS_SENHA`.

Por baixo dos panos: "deploy" é o ato de pegar seu código e colocá-lo num servidor acessível pela internet. A Vercel automatiza isso: cada `git push` vira uma nova versão publicada, sem você mexer em servidor. Se algo der errado, dá para voltar para uma versão anterior com um clique.

8. Boas práticas aplicadas neste projeto

- **Segredos fora do código** — senhas e URLs sensíveis ficam em variáveis de ambiente, nunca no Git.
- **Página não coleta dados pessoais** — respeita a LGPD; o rastreo usa apenas um código anônimo (`rid`).
- **Código separado por responsabilidade** — HTML, CSS e JS em arquivos diferentes.
- **Rastreo tolerante a falhas** — se o registro falhar, a página continua funcionando.
- **Acesso ao placar protegido por senha** — dados da campanha não ficam abertos a qualquer um.
- **Commits pequenos e descritivos** — o histórico do Git conta a evolução do projeto.

9. Glossário rápido

Termo	Significado simples
Phishing	Golpe digital que engana a pessoa para roubar senhas/dados, geralmente por e-mail ou site falso.
Front-end	A parte que o usuário vê e usa (a página no navegador).
Back-end	A parte que roda no servidor, invisível ao usuário (grava e devolve dados).
Deploy	Publicar o código para que fique acessível na internet.
Serverless Function	Um pedaço de back-end que a Vercel liga só quando é chamado.
Redis	Banco de dados muito rápido, ótimo para contadores.
rid	Código anônimo que identifica cada destinatário do e-mail (vem do GoPhish).
Variável de ambiente	Segredo (senha, endereço) guardado fora do código, no painel do serviço.
GoPhish	Ferramenta que dispara os e-mails de simulação de phishing.